

Magnetic Field Finger-Tracking Musical Glove

Group 7

B11202010 Yu-Min Han

B11901188 Yi-En Wu

B11901193 Yu-Tzu Liu

Department of Electrical Engineering, National Taiwan University

June 18, 2026

Abstract

This project implements a magnetic-field-based piano interface on the DE2-115 FPGA platform. Two AC electromagnets, driven at 75 Hz and 45 Hz, are used as distinguishable finger markers. Three QMC5883P magnetometers observe the resulting magnetic field, and the FPGA performs magnetometer readout, calibration, frequency-selective lock-in extraction, H^2 computation, LUT-based key classification, and stepper-motor platform tracking. The PC is only used as the final user interface: it receives FPGA-computed key and press states through RS232/UART, highlights the detected piano keys, and plays the corresponding tones. The final system demonstrates that magnetic sensing can provide a practical non-visual interface for localizing finger position and controlling a musical keyboard-like interaction.

Contents

1	Introduction	4
1.1	Project Goals	4
1.2	Final Scope	4
2	System Overview	6
2.1	Runtime Data Path	6
2.2	Major Subsystems	7
2.3	Local and Global Key Coordinates	7
3	Hardware Implementation	8
3.1	Magnetic Field Generation	8
3.2	Magnetometer Array	9
3.3	Stepper Motor and Sliding Platform	10
3.4	Press Buttons	10
4	FPGA Architecture	12
4.1	Module Overview	12
4.2	Sensor Readout	12
4.3	Magnetometer Calibration	13

4.4	Streaming Control Style	13
5	Frequency Separation and Signal Processing	14
5.1	Dual-Frequency Encoding	14
5.2	Digital Lock-In Filtering	14
5.3	Why the Lock-In Window Filters the Signal	15
5.4	Window Length Trade-Off	15
5.5	Additional H^2 Averaging	15
6	LUT-Based Key Classification	16
6.1	Motivation	16
6.2	LUT Construction	16
6.3	Normalized Feature Extraction	17
6.4	Why Normalization Helps Position Classification	17
6.5	Error Function	18
6.6	FPGA LUT ROM Format	18
6.7	Classifier State Machine Implementation	19
7	Platform Tracking and Global Key Mapping	19
7.1	Need for a Sliding Platform	19
7.2	Local-to-Global Conversion	19
7.3	Two-Finger Centering Rule	20
7.4	Motor Control Parameters	20
7.5	Stability and Cooldown	21
7.6	Tracker Implementation	21
8	PC Interface and Sound Output	22
8.1	UART Frame Format	22
8.2	Role of the PC	22
8.3	Piano GUI and Sound	22
9	Results and Discussion	23
9.1	Demonstrated Functions	23
9.2	Classification Accuracy Observations	23
9.3	Latency and Stability	24
9.4	Hardware and Debugging Challenges	24
10	Conclusion	24
10.1	Future Work	25
	Appendix A Pin Connections	25

Appendix B	UART Frame Reference	25
Appendix C	Important Runtime Parameters	26
Appendix D	Code Organization	26
Appendix E	Project File Structure	27

1 Introduction

Traditional piano keyboards and many touch interfaces rely on direct electrical or mechanical contact. For this final project, we explored a different sensing method: magnetic-field localization. The original motivation came from Finexus, a finger-tracking system that uses frequency-multiplexed electromagnets and multiple magnetic sensors to track fingertip motion [1]. Instead of attempting full 3D reconstruction in software, our project focuses on a digital-circuit implementation of a practical musical interface: determine which piano key a magnetic finger is over, move the sensing platform when needed, and output the corresponding musical note.

The final system is a magnetic-field finger-tracking musical glove. Two electromagnets are treated as two fingers. They are driven at different AC frequencies, 75 Hz and 45 Hz, so that their magnetic fields can be separated by frequency. Three QMC5883P magnetometers measure the magnetic field around a local sensing region. The FPGA performs the real-time localization pipeline and sends only the final key state to a PC. The PC does not perform localization; it only receives the FPGA-computed key indices and press states, displays a piano-key interface, and plays the notes.

1.1 Project Goals

The main goals of the project are:

- Build a real-time magnetic-field sensing system using the DE2-115 FPGA board.
- Distinguish two magnetic fingers by operating the electromagnets at two different frequencies.
- Perform magnetometer calibration, frequency-selective filtering, H^2 computation, and key classification on FPGA.
- Use a measured LUT to classify the local key position from the magnetic-field pattern across sensors.
- Extend the local sensing range using a stepper-motor-driven sliding platform.
- Send only the final key and press information to a PC for visualization and sound output.

1.2 Final Scope

The final implementation uses three magnetometers and a five-key local sensing window. The full keyboard layout contains 15 global key positions. The platform can slide in one-key increments, so the local five-key window can cover different parts of the global keyboard. The FPGA performs all sensing and decision-making tasks. The PC is intentionally kept outside the localization loop to preserve the digital-lab focus of the project.

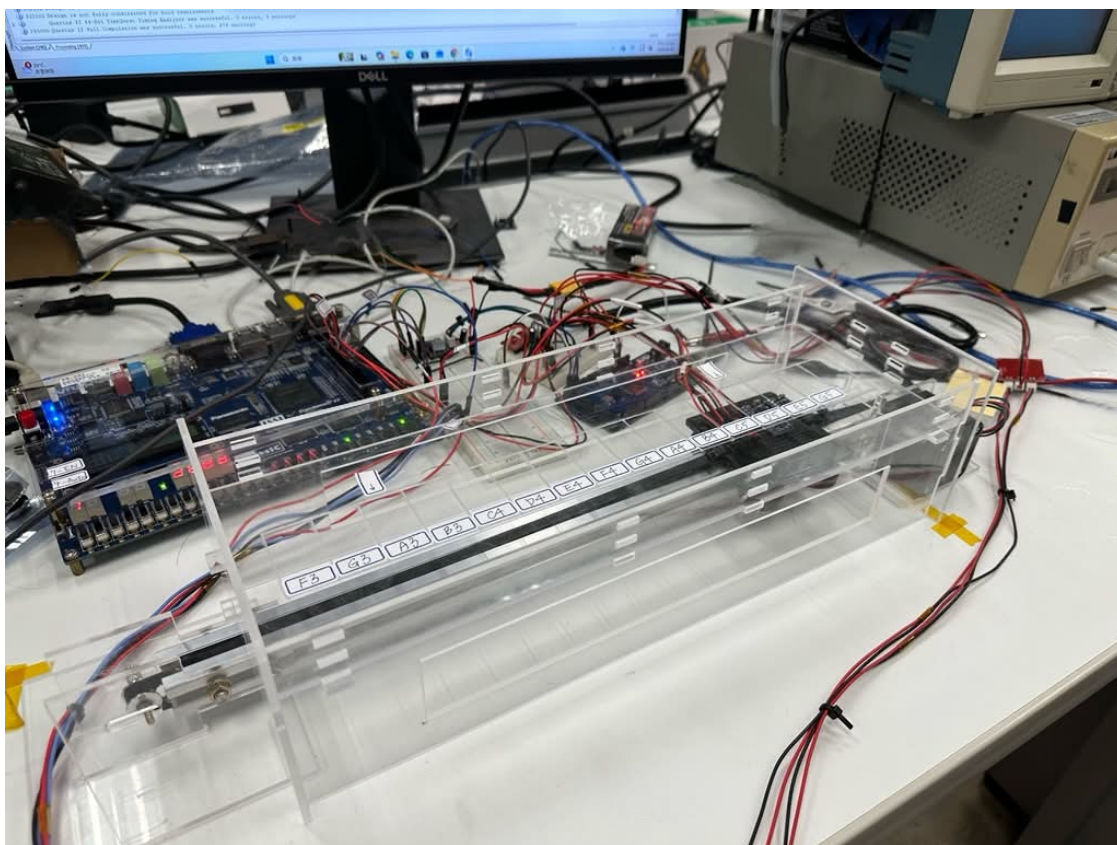


Figure 1: Final project hardware setup.

2 System Overview

Figure 2 shows the intended top-level data flow. The electromagnets generate AC magnetic fields. The magnetometers sense the field, and the FPGA converts raw magnetic samples into classified key positions. The sliding platform keeps the five-key sensing window near the active fingers. Finally, the PC receives key and press states and produces the visual and audio output.

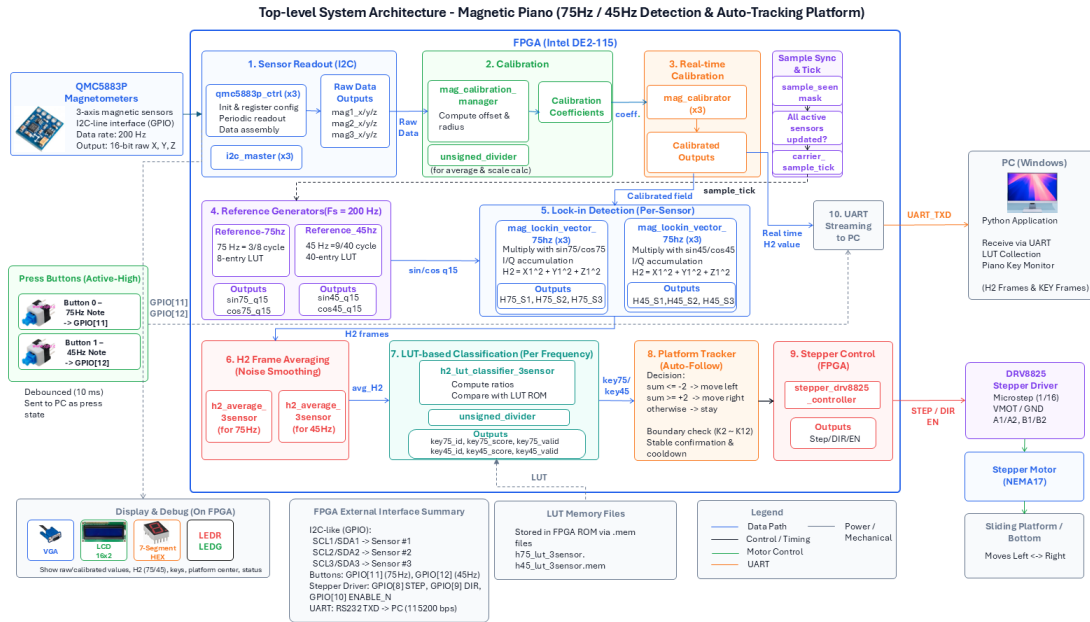


Figure 2: Top-level system architecture.

2.1 Runtime Data Path

The final runtime path is:

```

Electromagnets
-> QMC5883P magnetometers
-> FPGA I2C readout
-> FPGA calibration
-> FPGA 75/45 Hz lock-in filtering
-> FPGA H2 computation
-> FPGA LUT key classification
-> FPGA platform tracking and stepper control
-> UART key/press frame
-> PC piano GUI and sound output

```

Only the last step uses the PC. The FPGA calculates the magnetic features, classifies the keys, maintains the platform center, and directly drives the DRV8825 stepper driver.

2.2 Major Subsystems

Table 1: Major project subsystems.

Subsystem	Function	Implementation
Magnetic source	Generate distinguishable magnetic fields for two fingers	75 Hz and 45 Hz electromagnets
Sensor front end	Measure three-axis magnetic field	Three QMC5883P magnetometers
FPGA sensing pipeline	Read sensors, calibrate, filter, compute H^2	SystemVerilog modules on DE2-115
Key classifier	Compare live magnetic feature to LUT entries	FPGA ROM and fixed-point score calculation
Platform tracker	Move local sensing window across 15-key global layout	FPGA controller plus DRV8825 stepper driver
PC interface	Display keys and play notes	Python UART receiver and sound generator

2.3 Local and Global Key Coordinates

The classifier directly detects the key inside the current five-key local window. We represent the local index as

$$k_{\text{local}} \in \{-2, -1, 0, +1, +2\}, \quad (1)$$

where local index 0 is the key directly above the center of the sensor platform. The complete keyboard uses global indices

$$k_{\text{global}} \in \{0, 1, \dots, 14\}. \quad (2)$$

If the platform center is currently under global key k_{center} , then the global key is

$$k_{\text{global}} = k_{\text{center}} + k_{\text{local}}. \quad (3)$$

This relation is the core link between local magnetic classification and full-keyboard operation.

3 Hardware Implementation

3.1 Magnetic Field Generation

The project uses AC electromagnets instead of permanent magnets for the final two-finger system. The reason is that AC excitation allows frequency multiplexing: one finger can be driven at 75 Hz and the other at 45 Hz. The FPGA can then separate the two fingers by filtering at each frequency. This follows the same general principle used by Finexus, where multiple electromagnets are distinguished by their operating frequencies [1].

Each magnetic finger uses a hand-wound electromagnet. The coil is made by winding enameled copper wire around a ferrite core, with about 250 turns per electromagnet. The ferrite core increases the magnetic flux compared with an air-core coil, while the insulated enameled wire allows many tight turns without shorting adjacent windings. The number of turns was chosen as a practical compromise: more turns increase magnetomotive force for the same current, but also increase resistance, physical size, and heating.

The coil was designed with an impedance target of roughly $4\ \Omega$, matching the practical low-impedance load region of the TPA3110D2 audio power amplifier. If the coil impedance is too low, the amplifier and power supply must deliver excessive current, causing heat and possible protection shutdown. If the impedance is too high, the current becomes too small and the magnetometer signal amplitude decreases. The final 250-turn ferrite-core coil provided a usable magnetic field while keeping the amplifier load reasonable.

The analog magnetic-source chain is:

```
Arduino waveform table  
-> MCP4725 DAC  
-> TPA3110D2 power amplifier  
-> electromagnet coil
```

The Arduino and DAC are used only as waveform-generation hardware. They do not perform localization or key classification. The localization and control decisions are implemented on FPGA.



Figure 3: AC electromagnet driving chain.

3.2 Magnetometer Array

Three QMC5883P magnetometers are used in the final system. Each sensor is connected to an independent I2C-style bus implemented with FPGA GPIO pins. The independent buses avoid address conflicts and allow each sensor to be read with a dedicated controller instance.

Table 2: Magnetometer GPIO assignment.

Sensor	Signal	DE2 GPIO	FPGA Pin
1	SCL	GPIO[0]	PIN_AB22
1	SDA	GPIO[1]	PIN_AC15
2	SCL	GPIO[2]	PIN_AB21
2	SDA	GPIO[3]	PIN_Y17
3	SCL	GPIO[4]	PIN_AC21
3	SDA	GPIO[5]	PIN_Y16

The FPGA drives the SDA lines in an open-drain style: it actively drives low when needed and otherwise releases the line to high impedance. Pull-up resistors are required either on the breakout boards or externally.

3.3 Stepper Motor and Sliding Platform

The sensor platform is mounted on a sliding rail. A stepper motor controlled through a DRV8825 driver moves the platform by discrete one-key steps. This mechanical design allows a small local sensing window to cover a larger keyboard. The final measured relation is:

$$1 \text{ key} = 2 \text{ cm} = 100 \text{ motor steps.} \quad (4)$$

The FPGA sends three digital signals to the DRV8825:

Table 3: DRV8825 control signals.

Signal	DE2 GPIO	FPGA Pin	Meaning
STEP	GPIO[8]	PIN_AD15	A rising edge advances one step or microstep.
DIR	GPIO[9]	PIN_AE15	Selects motor direction.
ENABLE_N	GPIO[10]	PIN_AC19	Active-low driver enable.

3.4 Press Buttons

Two active-high press buttons are used to indicate whether the detected finger is actually pressing a key. Their signals are debounced on FPGA for 10 ms before being transmitted to the PC.

Table 4: Press button assignment.

Button	DE2 GPIO	FPGA Pin	Meaning
Button 0	GPIO[11]	PIN_AF16	Press state for the 75 Hz tracked note.
Button 1	GPIO[12]	PIN_AD19	Press state for the 45 Hz tracked note.

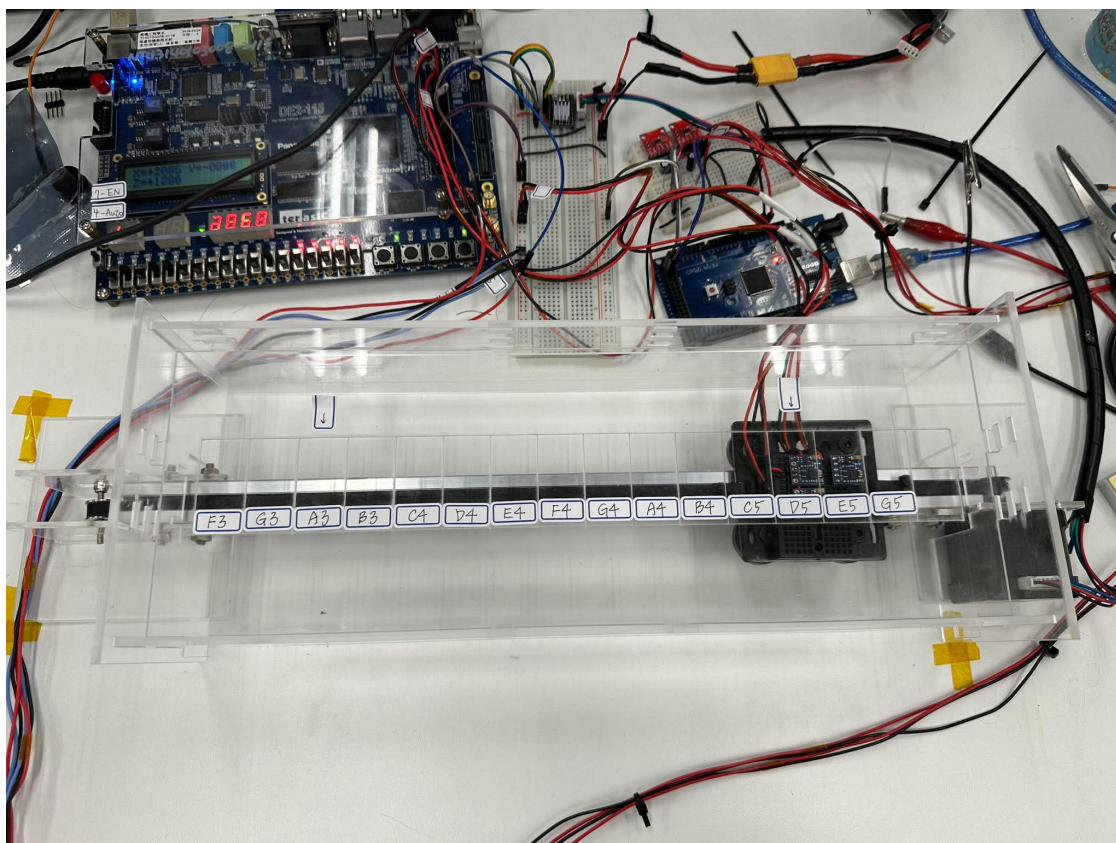


Figure 4: Sliding platform and sensing window.

4 FPGA Architecture

The FPGA design is implemented in SystemVerilog and organized as a set of dedicated modules. The top-level file is `DE2_115.sv`. The final design uses the FPGA for all real-time localization and control computations.

4.1 Module Overview

Table 5: Important FPGA modules.

Module	Function
<code>qmc5883p_ctrl.sv</code>	Initializes and reads one QMC5883P magnetometer through I2C.
<code>i2c_master.sv</code>	Low-level I2C transaction controller.
<code>mag_calibration_manager.sv</code>	Collects min/max samples and generates offset/scale coefficients.
<code>mag_calibrator.sv</code>	Applies offset and scale correction to raw magnetometer data.
<code>carrier_reference_75hz.sv</code>	Generates sine/cosine reference samples for 75 Hz extraction.
<code>carrier_reference_45hz.sv</code>	Generates sine/cosine reference samples for 45 Hz extraction.
<code>mag_lockin_vector_75hz.sv</code>	Performs rolling-window lock-in filtering and vector H^2 calculation.
<code>h2_average_3sensor.sv</code>	Averages the latest 10 complete H^2 frames before classification.
<code>h2_lut_classifier_3sensor.sv</code>	Compares live normalized H^2 features with LUT ROM entries.
<code>platform_tracker_controller.sv</code>	Converts local key decisions into one-key platform movement commands.
<code>stepper_drv8825_controller.sv</code>	Generates STEP, DIR, and ENABLE_N signals for the DRV8825.
<code>mag_uart_streamer.sv</code>	Streams H^2 and final key/press frames to the PC.

4.2 Sensor Readout

Each QMC5883P controller checks the chip ID, configures the sensor for continuous measurement, and repeatedly reads the X, Y, and Z magnetic-field registers. In the current configuration, the sensor is set to continuous mode, 200 Hz output data rate, and ± 2 G range. The FPGA receives three signed 16-bit raw values from each sensor.

The controller emits a one-clock `sample_valid` pulse after each fresh sensor sample. Since the three I2C controllers do not necessarily finish in the same clock cycle, the top-level design collects valid pulses into a mask. A new lock-in sample is accepted only after all active sensors have produced a new sample.

In implementation, the three sensor controllers run in parallel instead of being multiplexed through one shared I2C controller. The top-level `DE2_115.sv` module latches each sensor result when its valid pulse arrives, then clears the collection mask only after all three sensors have

updated. This prevents the lock-in filter from mixing a new sample from one sensor with old samples from the other sensors.

4.3 Magnetometer Calibration

The calibration stage compensates for sensor offset and axis-scale mismatch. During calibration, the user rotates the complete sensor assembly. The FPGA records the maximum and minimum value for each active sensor axis:

$$\text{offset} = \frac{\text{max} + \text{min}}{2}. \quad (5)$$

The measured radius of each axis is

$$r_{\text{axis}} = \frac{\text{max} - \text{min}}{2}. \quad (6)$$

The scale coefficient is derived by comparing each axis radius to a target radius:

$$\text{scale}_{\text{axis}} = \frac{r_{\text{target}}}{r_{\text{axis}}}. \quad (7)$$

The calibrated output is then

$$B_{\text{cal}} = (B_{\text{raw}} - \text{offset}) \cdot \text{scale}. \quad (8)$$

This correction is implemented on FPGA. No Python calibration is needed during the final runtime.

4.4 Streaming Control Style

Most of the FPGA pipeline is controlled by one-clock valid pulses. A module latches its input when the upstream `valid` signal is asserted, performs its internal multi-cycle calculation if needed, then emits a new `valid` pulse for the next stage. This style is used for sensor readout, lock-in output, H^2 averaging, LUT classification, platform tracking, and UART streaming. It keeps the modules independent while still allowing slower blocks, such as division and ROM scanning, to take multiple clock cycles.

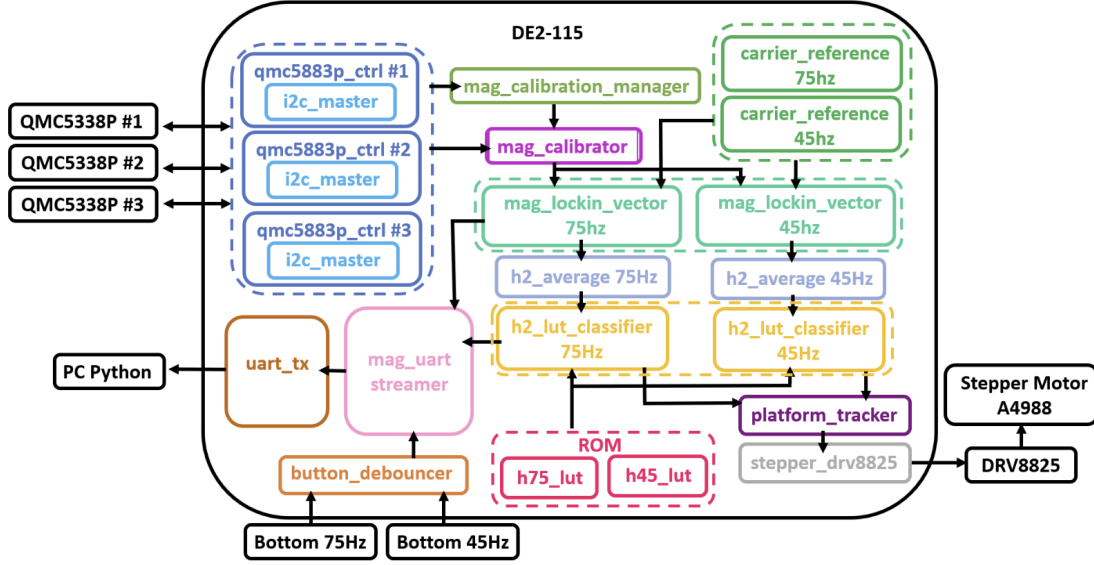


Figure 5: FPGA datapath architecture.

5 Frequency Separation and Signal Processing

5.1 Dual-Frequency Encoding

The two electromagnets are driven at different frequencies:

$$f_1 = 75 \text{ Hz}, \quad f_2 = 45 \text{ Hz}. \quad (9)$$

This frequency separation allows the FPGA to identify the magnetic contribution of each finger even though all sensors measure the sum of both fields plus ambient noise. The FPGA extracts the energy at each target frequency independently.

5.2 Digital Lock-In Filtering

For each sensor and each target frequency, the FPGA performs a digital lock-in operation. The calibrated magnetic signal is multiplied by a reference cosine and sine:

$$I[n] = x[n] \cos(\omega n), \quad (10)$$

$$Q[n] = x[n] \sin(\omega n). \quad (11)$$

The products are accumulated over a rolling window. For one axis, the accumulated in-phase and quadrature components are:

$$I_{\text{acc}} = \sum_{n=0}^{N-1} x[n] \cos(\omega n), \quad (12)$$

$$Q_{\text{acc}} = \sum_{n=0}^{N-1} x[n] \sin(\omega n), \quad (13)$$

where $N = \text{WINDOW_SAMPLES} = 100$ in the final design.

The same operation is applied to the X, Y, and Z axes. The lock-in output energy for one sensor is:

$$H^2 = X_I^2 + X_Q^2 + Y_I^2 + Y_Q^2 + Z_I^2 + Z_Q^2. \quad (14)$$

The module `mag_lockin_vector_75hz.sv` implements this as a rolling coherent window. For every accepted sample, the FPGA multiplies each calibrated axis by the Q15 sine and cosine references, stores the six products in window memory, adds the new products to the accumulators, and subtracts the products leaving the window. The product history arrays are marked for M9K RAM inference so the 100-sample window does not consume a large number of logic registers. After the accumulators are updated, a small state machine squares the six accumulated values one at a time, sums them, applies the fixed scaling shift, saturates on overflow, and asserts `result_valid`.

5.3 Why the Lock-In Window Filters the Signal

When the input signal contains the same frequency as the reference, the product accumulates coherently. When the signal is DC, noise, or another frequency, the positive and negative products tend to cancel. Therefore, the 75 Hz extractor responds strongly to the 75 Hz electromagnet and weakly to the 45 Hz electromagnet. The 45 Hz extractor behaves similarly for the other finger.

5.4 Window Length Trade-Off

The final design uses a 100-sample rolling lock-in window. A larger window improves frequency selectivity and noise rejection, but it also increases latency. A smaller window responds faster, but it gives less separation between 45 Hz, 75 Hz, DC, and noise. This trade-off is important because the key classifier receives the filtered H^2 values.

Table 6: Effect of lock-in window size.

Window size	Advantage	Disadvantage
Larger	Better noise rejection and frequency separation	Slower response after movement
Smaller	Faster response	Noisier H^2 and weaker frequency rejection

5.5 Additional H^2 Averaging

After the lock-in stage, the FPGA applies a second, simpler rolling average before classification. The module `h2_average_3sensor.sv` averages the latest 10 complete H^2 frames for each frequency. This reduces classification jitter and matches the Python prototype’s default averaging behavior. The resulting path is:

```
100-sample lock-in window
```

```
-> H^2 output
-> 10-frame H^2 rolling average
-> LUT classifier
```

This averaging improves stability but can briefly retain old samples after the platform moves. This effect is discussed in Section 9.

The averaging module uses the same rolling-sum idea as the lock-in filter. It keeps a 10-entry circular buffer for each sensor, subtracts the overwritten value, adds the newest H^2 value, and divides the maintained sum by 10 when the window is full. Therefore each new averaged output is produced with constant work instead of recomputing the full 10-sample sum from scratch.

6 LUT-Based Key Classification

6.1 Motivation

The final system does not solve a full continuous 3D inverse magnetic-field equation during runtime. Instead, it uses a measured lookup table (LUT). This approach is practical for our application because the piano keys form a discrete set of positions. The classifier only needs to determine which key is most likely, not estimate a continuous 3D coordinate.

Each key position produces a characteristic distribution of H^2 across the three sensors. The FPGA compares the live distribution with stored LUT entries and selects the key with the smallest error.

6.2 LUT Construction

For each frequency, a separate LUT is collected:

- `h75_lut.csv` for the 75 Hz finger.
- `h45_lut.csv` for the 45 Hz finger.

The local sensing window contains five keys:

$$k_{\text{local}} \in \{-2, -1, 0, +1, +2\}. \quad (15)$$

For each key, three horizontal sample points are collected: top, middle, and bottom. To improve robustness to different finger heights, three vertical layers are collected: the key plane and two hover planes. Thus, one frequency LUT contains:

$$5 \text{ keys} \times 3 \text{ points/key} \times 3 \text{ layers} = 45 \text{ entries}. \quad (16)$$

Each entry is an average of multiple received H^2 frames, so the LUT represents a stable measured magnetic pattern.

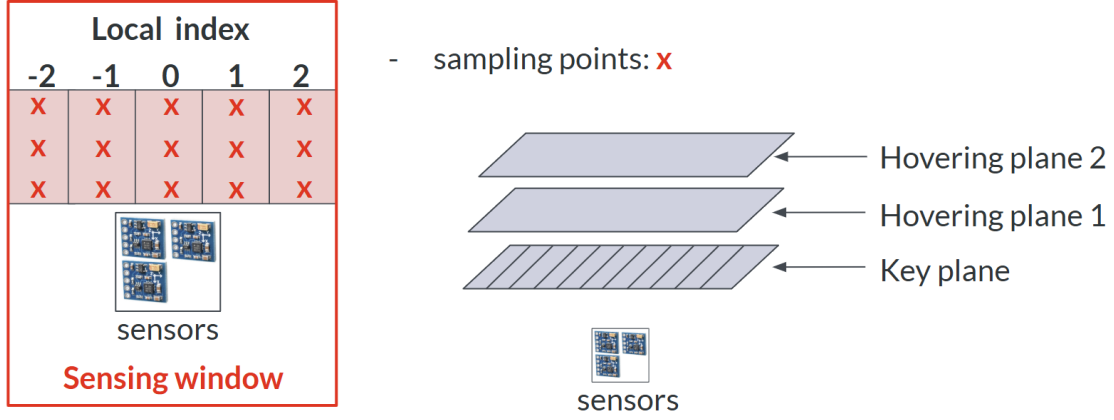


Figure 6: LUT sampling positions.

6.3 Normalized Feature Extraction

Directly comparing raw H^2 values is sensitive to total field strength. If the finger is slightly closer or farther from the sensors, all H^2 values can change even when the position is the same. To focus on position, the classifier uses normalized sensor ratios:

$$H_{\text{total}}^2 = H_1^2 + H_2^2 + H_3^2, \quad (17)$$

$$f_1 = \frac{H_1^2}{H_{\text{total}}^2}, \quad (18)$$

$$f_2 = \frac{H_2^2}{H_{\text{total}}^2}, \quad (19)$$

$$f_3 = \frac{H_3^2}{H_{\text{total}}^2}. \quad (20)$$

The ratios satisfy

$$f_1 + f_2 + f_3 = 1. \quad (21)$$

This means the classifier compares which sensor is strongest relative to the others, rather than comparing only absolute magnitude.

6.4 Why Normalization Helps Position Classification

Raw H^2 mainly describes signal strength. Normalized H^2 describes the spatial pattern across sensors. For example, when the magnetic finger is closer to sensor 1, sensor 1 becomes dominant. When it moves toward sensor 2, sensor 2 becomes more dominant. This relative distribution is more directly related to position.

A simple example is shown in Table 7. The live sample is actually Key A. A raw H^2 comparison would incorrectly choose Key B because the absolute values are closer to Key B. However, after normalization, the live pattern exactly matches Key A.

Table 7: Example showing why normalized H^2 improves position classification.

Sample	H_1^2	H_2^2	H_3^2	Normalized pattern
LUT Key A	10	5	5	[0.50, 0.25, 0.25]
LUT Key B	18	9	3	[0.60, 0.30, 0.10]
Live at Key A	20	10	10	[0.50, 0.25, 0.25]

The raw squared error to Key A is

$$(20 - 10)^2 + (10 - 5)^2 + (10 - 5)^2 = 150. \quad (22)$$

The raw squared error to Key B is

$$(20 - 18)^2 + (10 - 9)^2 + (10 - 3)^2 = 54. \quad (23)$$

Raw H^2 therefore chooses Key B incorrectly. In contrast, the normalized error to Key A is zero, so normalized features correctly preserve the position pattern.

6.5 Error Function

The FPGA uses the same core scoring method as the Python prototype. For a live feature vector and one LUT entry:

$$E_{\text{ratio}} = (f_{1,\text{live}} - f_{1,\text{LUT}})^2 + (f_{2,\text{live}} - f_{2,\text{LUT}})^2 + (f_{3,\text{live}} - f_{3,\text{LUT}})^2, \quad (24)$$

$$E_{\text{strength}} = \left(\ln(H_{\text{total,live}}^2) - \ln(H_{\text{total,LUT}}^2) \right)^2, \quad (25)$$

$$E_{\text{final}} = E_{\text{ratio}} + 0.05E_{\text{strength}}. \quad (26)$$

The logarithmic strength term keeps useful distance information without allowing raw magnitude to dominate the score.

6.6 FPGA LUT ROM Format

The LUT is stored in FPGA ROM initialized from a hexadecimal `.mem` file. Each line is one 160-bit LUT entry:

```
[159:156] key_id
[155:128] reserved
[127:96] f1_q30
[95:64] f2_q30
[63:32] f3_q30
[31:0] ln_total_q16
```

The ratio values are stored in Q30 fixed-point format:

$$f_{q30} = f \cdot 2^{30}. \quad (27)$$

The strength term is stored as Q16:

$$\ln(H_{\text{total}}^2)_{q16} = \ln(H_{\text{total}}^2) \cdot 2^{16}. \quad (28)$$

During classification, the FPGA scans all 45 LUT entries. For each key, it keeps the minimum entry score among all sample points and z layers. The predicted local key is the key with the lowest final score.

6.7 Classifier State Machine Implementation

The classifier is implemented in `h2_lut_classifier_3sensor.sv` as a multi-cycle finite-state machine. When `start` is asserted, the module latches the three live H^2 values and computes their total. It then uses the shared `unsigned_divider.sv` block three times to calculate the Q30 ratios f_1 , f_2 , and f_3 . The logarithmic strength feature is approximated in hardware by finding the base-2 exponent of the total strength and adding a small mantissa lookup correction in Q16.

After the live feature vector is ready, the classifier reads the 160-bit ROM entries initialized from the `.mem` LUT file. For each entry, it extracts the key ID, three Q30 ratio fields, and Q16 log-strength field. The state machine computes absolute differences, squares one component per cycle, adds the three ratio scores, adds the weighted strength score, and updates the best score stored for that key. After all 45 entries have been scanned, a final pass compares the five per-key best scores and outputs the minimum-score key with a one-clock `valid` pulse.

7 Platform Tracking and Global Key Mapping

7.1 Need for a Sliding Platform

The three-sensor array only provides reliable classification over a local five-key window. However, the desired piano layout contains 15 global keys. To extend the effective range without adding more sensors, the sensor platform is mounted on a stepper-driven rail. The FPGA moves the platform so that the active fingers remain near the center of the local sensing window.

7.2 Local-to-Global Conversion

The local key index is

$$k_{\text{local}} \in \{-2, -1, 0, +1, +2\}. \quad (29)$$

The global key index is

$$k_{\text{global}} \in \{0, 1, \dots, 14\}. \quad (30)$$

If the platform center is at global key k_{center} , then

$$k_{\text{global}} = k_{\text{center}} + k_{\text{local}}. \quad (31)$$

For example, if the platform is centered at global key 12, then the five local indices map to:

Table 8: Example local-to-global mapping for platform center 12.

k_{local}	-2	-1	0	+1	+2
k_{global}	10	11	12	13	14

7.3 Two-Finger Centering Rule

The 75 Hz finger is assumed to be the left finger and the 45 Hz finger is assumed to be the right finger. After classification, the FPGA obtains two local indices:

$$k_{75}, k_{45} \in \{-2, -1, 0, +1, +2\}. \quad (32)$$

The controller computes:

$$S = k_{75} + k_{45}. \quad (33)$$

The movement rule is:

$$\begin{cases} S \leq -2, & \text{move platform left by one key,} \\ S \geq +2, & \text{move platform right by one key,} \\ -2 < S < +2, & \text{stay.} \end{cases} \quad (34)$$

This rule keeps the center of the two fingers near the center of the sensing platform.

7.4 Motor Control Parameters

The measured mechanical calibration is:

$$1 \text{ key} = 100 \text{ steps}. \quad (35)$$

The final FPGA stepper controller uses:

- Step rate: 800 steps/s.
- STEP high pulse width: 5 μ s.
- Fixed movement: 100 steps per automatic one-key move.
- Allowed platform center: global keys 2 through 12.
- Initial platform center: global key 12.

7.5 Stability and Cooldown

To avoid reacting to momentary classification noise, the platform tracker requires the same movement request for three consecutive decisions:

$$\text{STABLE_COUNT} = 3. \quad (36)$$

After a movement completes, the tracker waits:

$$\text{COOLDOWN_MS} = 100 \text{ ms}. \quad (37)$$

This cooldown prevents immediate repeated movement. However, classification output can still be affected by stale samples in the lock-in and H^2 averaging windows immediately after movement. In practice, this can cause occasional ± 1 key errors if a press occurs immediately after the platform stops.

7.6 Tracker Implementation

The module `platform_tracker_controller.sv` latches the latest 75 Hz and 45 Hz classifier results separately. A movement decision is evaluated only after both fingers have produced fresh classifications, so the tracker does not combine a new key from one finger with an old key from the other. The local key IDs from the classifier are converted to signed local indices by subtracting the local center ID 2, giving the range -2 to $+2$.

Before issuing a move, the controller checks several blocking conditions: tracking must be enabled, both signal strengths must exceed the minimum threshold, the 75 Hz finger must not appear to the right of the 45 Hz finger, the stepper must not already be busy, the cooldown counter must be zero, and the platform center must remain within keys 2 through 12. If the requested direction passes these checks for `STABLE_COUNT=3` consecutive fresh decisions, the tracker emits one `move_pulse`, sets the direction command, and updates the stored platform center by one key. The separate `stepper_drv8825_controller.sv` module then converts that pulse into 100 physical STEP pulses.

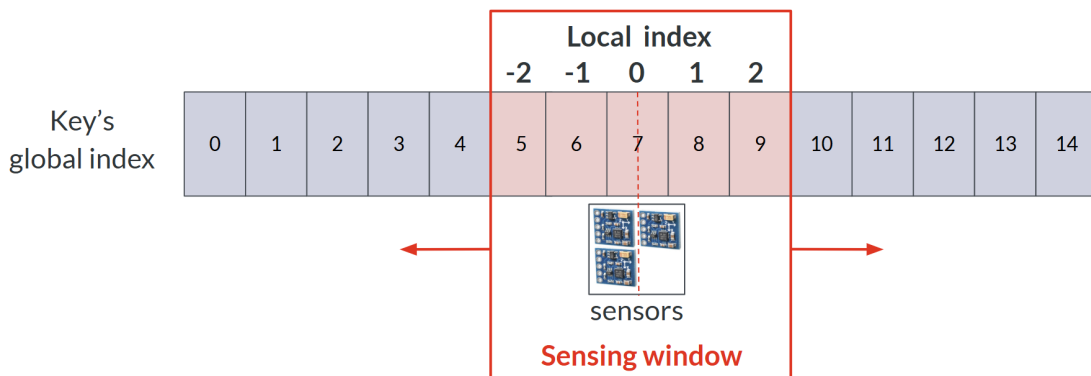


Figure 7: Sliding platform extends local sensing to global key tracking.

8 PC Interface and Sound Output

8.1 UART Frame Format

The FPGA streams compact ASCII frames to the PC through RS232/UART. Two frame types are used. The H^2 frame is mainly useful for debugging and LUT collection:

```
H2,75,H75S1,H75S2,H75S3,45,H45S1,H45S2,H45S3
```

The final runtime piano interface uses the key frame:

```
KEY,75,KEY75,PRESS75,45,KEY45,PRESS45
```

KEY75 and KEY45 are two-digit global key indices, from 00 to 14. PRESS75 and PRESS45 are debounced active-high button states.

8.2 Role of the PC

The PC does not perform magnetic localization in the final system. Its responsibilities are:

- Receive the FPGA key/press frame.
- Map global key indices to note names.
- Display a live piano-key GUI.
- Play or stop the corresponding sound when the FPGA press bit changes.

This division keeps the project focused on FPGA computation. The Python script is an output interface, not a classifier.

8.3 Piano GUI and Sound

The final Python monitor maps global key indices to musical notes. The command used for the 15-key layout is:

```
python3 src_python/piano_key_monitor.py COM5 \
--notes F3,G3,A3,B3,C4,D4,E4,F4,G4,A4,B4,C5,D5,E5,F5 \
--buffer-size 64 --poll-ms 2 --volume 0.6
```

The monitor starts a note when the corresponding press bit becomes 1 and stops it when the press bit returns to 0. With the pygame audio backend, the sound is played as a held note with low-latency start and exact stop on release.

In `piano_key_monitor.py`, serial reception runs in a background thread so the Tkinter GUI does not block while waiting for UART bytes. The reader thread accepts only lines matching the `KEY,75,...,45,...` format and pushes parsed events into a queue. The GUI periodically polls this queue, updates the highlighted keys, and calls the audio backend before redrawing the canvas so note start and stop are not delayed by screen updates.

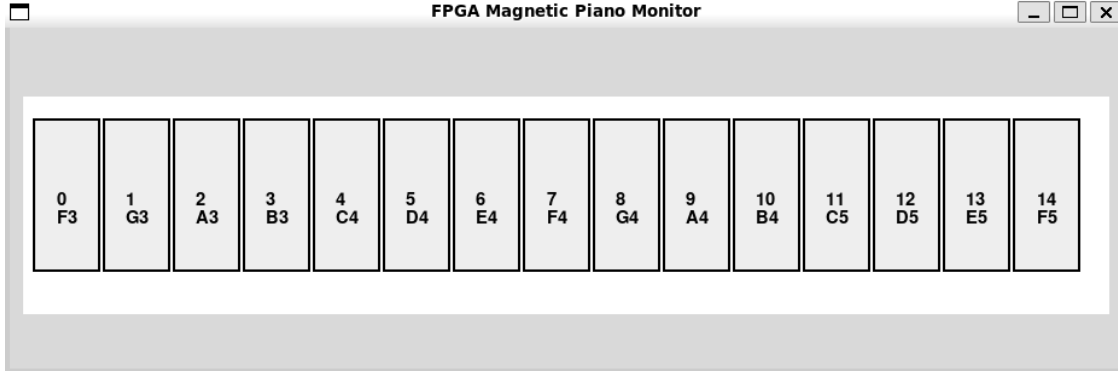


Figure 8: PC piano GUI receiving FPGA-computed key states.

9 Results and Discussion

9.1 Demonstrated Functions

The final system demonstrates the following functions:

- Three QMC5883P magnetometers are read continuously by the FPGA through independent I2C buses.
- The FPGA calibrates magnetometer offsets and scale mismatch.
- The FPGA separates 75 Hz and 45 Hz magnetic components through digital lock-in filtering.
- The FPGA computes H^2 values for each sensor and each frequency.
- The FPGA classifies local key position with measured LUT ROMs.
- The FPGA converts local key results into global key indices using the current platform center.
- The FPGA controls the DRV8825 stepper driver to move the sensing platform.
- The PC receives only the final key/press state and plays notes.

9.2 Classification Accuracy Observations

During development, Python classification using the measured LUT produced good key classification results. The first FPGA classifier did not initially match the Python behavior because the error formula was not identical. After modifying the FPGA to use the same normalized-ratio score and the same logarithmic strength term, the FPGA classification became significantly more accurate.

The final feature vector is:

$$[f_1, f_2, f_3, \ln(H_{\text{total}}^2)]. \quad (38)$$

This feature performed better than direct raw H^2 comparison because the normalized ratios preserve spatial pattern while reducing sensitivity to total magnitude variation.

9.3 Latency and Stability

Several averaging and filtering stages affect response time:

1. The lock-in filter uses a 100-sample rolling coherent window.
2. The classifier input uses a 10-frame rolling H^2 average.
3. The platform tracker requires three stable movement decisions before moving.
4. The tracker uses a 100 ms cooldown after stepper movement.

These stages improve stability and noise rejection, but they also introduce delay. The most visible effect is after platform movement: the H^2 average and lock-in windows can still contain samples from before the platform moved. If a key is pressed immediately after the platform finishes moving, the classification may occasionally be off by one key. Waiting briefly after movement gives the rolling windows time to fill with samples from the new platform position.

9.4 Hardware and Debugging Challenges

Several practical issues shaped the final design:

- **Sensor reliability:** At one point, a replacement magnetometer behaved abnormally and caused intermittent output freezing. Replacing the sensor solved the problem.
- **FPGA resource usage:** Early filtering implementations used too many logic resources. The lock-in product history was moved to M9K-style RAM storage to reduce LAB usage.
- **Raw versus normalized features:** Raw H^2 values could be misleading because magnitude changes dominate the error. Normalized ratios plus a small strength term produced better classification.
- **Stepper motor behavior:** Abrupt direction changes can cause vibration or missed motion. The final controller uses stable-decision gating and cooldown to reduce unnecessary moves.
- **Audio latency:** The Python interface originally played short fixed clips. It was later changed to held-note playback so the sound starts and stops with the FPGA press state.

10 Conclusion

This project demonstrates a magnetic-field-based piano interface in which the FPGA performs the main real-time computation. The final system reads three magnetometers, calibrates their outputs, extracts 75 Hz and 45 Hz magnetic-field strength using lock-in filtering, computes H^2 , classifies key position using FPGA LUT ROMs, controls a stepper-driven sliding platform, and sends final key/press states to a PC for sound output.

The main technical result is the FPGA-side LUT classifier. By using normalized H^2 ratios and a logarithmic strength term, the classifier compares the spatial magnetic pattern instead of only the absolute field magnitude. This made the FPGA result consistent with the Python prototype and improved key classification accuracy.

The sliding platform further extends the sensing range. A five-key local magnetic sensing window can be mapped to a 15-key global keyboard using the platform center state. The formula

$$k_{\text{global}} = k_{\text{center}} + k_{\text{local}} \quad (39)$$

allows the FPGA to report global key positions while the mechanical platform moves by one-key increments.

10.1 Future Work

Future improvements include:

- Resetting or flushing the lock-in and H^2 averaging windows after platform movement to reduce immediate post-move classification errors.
- Adding acceleration and deceleration profiles for smoother stepper motor motion.
- Expanding the LUT or using interpolation for smoother position estimation.
- Adding more sensors to improve robustness and coverage.
- Generating audio directly from FPGA instead of relying on the PC for sound output.
- Integrating a cleaner mechanical keyboard frame for repeatable finger height and key position.

Overall, the project shows that magnetic-field sensing can be combined with FPGA signal processing to build an interactive musical input device with real-time classification and platform control.

Appendix A Pin Connections

Table 9: External pin summary.

Connection	DE2 GPIO	Description
Sensor 1 SCL/SDA	GPIO[0]/GPIO[1]	Independent I2C bus for magnetometer 1.
Sensor 2 SCL/SDA	GPIO[2]/GPIO[3]	Independent I2C bus for magnetometer 2.
Sensor 3 SCL/SDA	GPIO[4]/GPIO[5]	Independent I2C bus for magnetometer 3.
DRV8825 STEP	GPIO[8]	Step pulse output.
DRV8825 DIR	GPIO[9]	Direction output.
DRV8825 EN-ABLE_N	GPIO[10]	Active-low driver enable.
75 Hz press button	GPIO[11]	Active-high press state for 75 Hz finger.
45 Hz press button	GPIO[12]	Active-high press state for 45 Hz finger.

Appendix B UART Frame Reference

H2,75,H75S1,H75S2,H75S3,45,H45S1,H45S2,H45S3
 KEY,75,KEY75,PRESS75,45,KEY45,PRESS45

The H^2 values are unsigned 32-bit hexadecimal Q16 values. The key indices are decimal global key indices. The press bits are active-high digital states after FPGA debouncing.

Appendix C Important Runtime Parameters

Table 10: Important final parameters.

Parameter	Value
Magnetometer count	3
Magnetometer output data rate	200 Hz
Magnetometer range	± 2 G
Finger frequencies	75 Hz and 45 Hz
Lock-in window	100 samples
Classifier H^2 average	10 frames
LUT entries per frequency	45
Local key range	-2 to +2
Global key range	0 to 14
One-key movement	100 motor steps
Stepper speed	800 steps/s
Step pulse width	5 μ s
Platform cooldown	100 ms

Appendix D Code Organization

The main FPGA source files are located in `src/`. The main Python utility files are located in `src_python/`. The final PC piano interface is `src_python/piano_key_monitor.py`. The FPGA LUT ROM files are `src/h75_lut_3sensor.mem` and `src/h45_lut_3sensor.mem`.

Appendix E Project File Structure

The submitted repository is organized as follows:

```
team07_final/
|-- README.md
|-- presentation_slides.pdf
|-- report.pdf
|-- src/
|   |-- DE2_115.qsf
|   |-- DE2_115.sv
|   |-- button_debouncer.sv
|   |-- carrier_reference_45hz.sv
|   |-- carrier_reference_75hz.sv
|   |-- h2_average_3sensor.sv
|   |-- h2_lut_classifier_3sensor.sv
|   |-- h45_lut_3sensor.mem
|   |-- h75_lut_3sensor.mem
|   |-- i2c_master.sv
|   |-- lcd_ctrl.sv
|   |-- mag_calibration_manager.sv
|   |-- mag_calibrator.sv
|   |-- mag_lockin_vector_75hz.sv
|   |-- mag_magnitude_squared.sv
|   |-- mag_uart_streamer.sv
|   |-- platform_tracker_controller.sv
|   |-- qmc5883p_ctrl.sv
|   |-- stepper_drv8825_controller.sv
|   |-- uart_tx.sv
|   |-- unsigned_divider.sv
|   |-- vga_font_rom.sv
|   |-- vga_sensor1_dashboard.sv
|   '-- vga_timing_640x480.sv
'-- src_python/
    |-- animate_h2_keys.py
    |-- classify_h2_lut.py
    |-- collect_h2_lut.py
    |-- export_lut_mem.py
    |-- h45_lut.csv
    |-- h45_lut_old.csv
    |-- h75_lut.csv
    |-- h75_lut_old.csv
    |-- piano_key_monitor.py
    |-- read_key_uart.py
    |-- read_mag_uart.py
    '-- track_platform_stepper.py
```

The Verilog/SystemVerilog files in `src/` implement the FPGA runtime system. The Python files in `src_python/` are used for LUT collection, LUT export, debugging, and the final PC piano interface.

References

- [1] K.-Y. Chen, S. Patel, and S. Keller, “Finexus: Tracking precise motions of multiple fingertips using magnetic sensing,” in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, pp. 1504–1514, ACM, 2016.